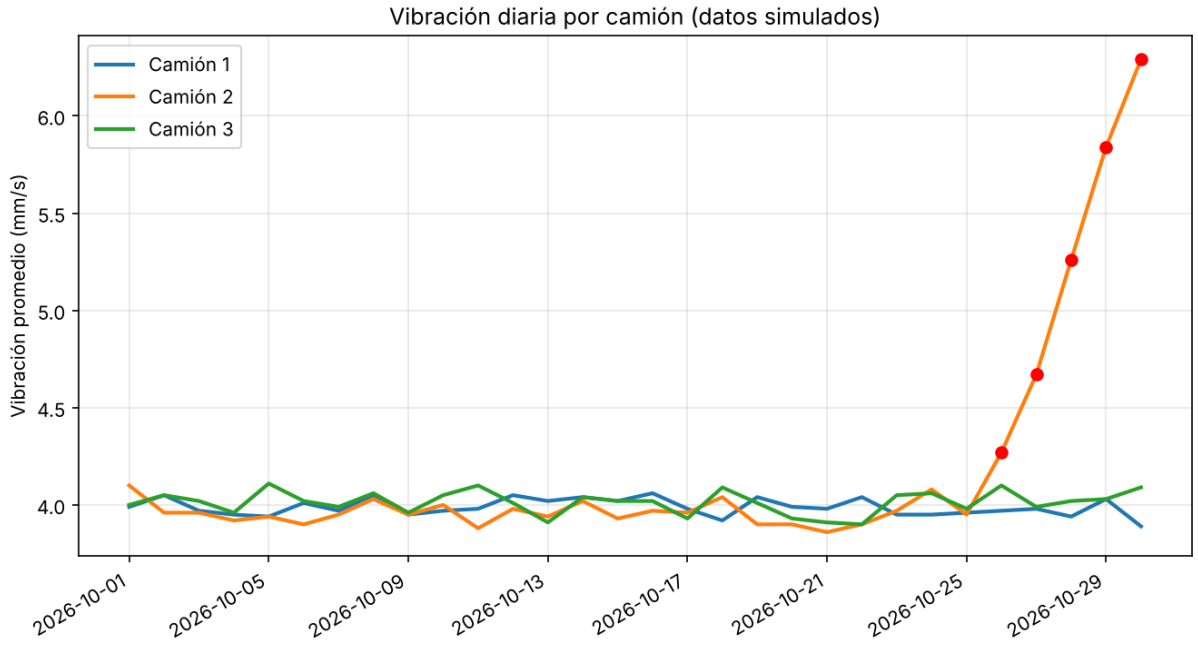
Cómo armé el ejemplo de los sensores en camiones

Guía paso a paso del reel 3, hecha en Python y SQL. Los datos son simulados: los generé yo para practicar. Preparada por Matías, estudiante de Ciencia de Datos de San Juan.

1. Qué vas a lograr

Vas a simular los sensores de tres camiones mineros durante 30 días, cargarlos con pandas, resumirlos con SQL, detectar automáticamente un camión que se deteriora y graficar el resultado.



El camión 2 empieza a deteriorarse el día 25 y el sistema marca la primera alerta el 26 de octubre, un día después.

2. Antes de empezar

Necesitás Python 3.9 o más nuevo. Instalá las librerías:

```
pip install pandas numpy matplotlib
```

SQLite ya viene con Python, no hace falta instalar nada más.

3. La idea en cinco pasos

1. Simular las lecturas de los sensores.
2. Cargarlas con pandas.
3. Resumirlas por día con SQL.
4. Definir una regla para detectar lo anormal.
5. Graficar.

4. Paso 1: simular los datos

Cada camión tiene una lectura por hora durante 30 días, con dos sensores: vibración en mm/s y temperatura en grados. Los valores normales rondan 4 mm/s y 85 grados, con un poco de ruido al azar. Al camión 2 le sumamos un deterioro gradual desde el día 25.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)            # semilla: siempre da lo mismo
horas = pd.date_range("2026-10-01", periods=30 * 24, freq="h")

vib = rng.normal(4.0, 0.3, len(horas))      # vibración normal
temp = rng.normal(85, 2.0, len(horas))      # temperatura normal

# deterioro del camión 2 desde el día 25
inicio = 25 * 24
deterioro = np.linspace(0, 2.5, len(horas) - inicio)
vib[inicio:] += deterioro
temp[inicio:] += deterioro * 2
```

5. Paso 2: cargar con pandas

Guardamos las lecturas en un CSV y lo leemos como se haría con datos reales:

```
df = pd.read_csv("data/simulado/sensores_camiones.csv", parse_dates=["fecha_hora"])
print(df.head())
```

Columnas: `fecha_hora`, `camion`, `vibracion_mm_s`, `temperatura_c`.

6. Paso 3: resumir con SQL

Pasamos la tabla a SQLite y pedimos el promedio diario por camión:

```
import sqlite3

con = sqlite3.connect("data/mineria_demo.db")
df.to_sql("sensores", con, if_exists="replace", index=False)

diario = pd.read_sql("""
    SELECT camion,
           DATE(fecha_hora)             AS dia,
           ROUND(AVG(vibracion_mm_s),2) AS vibracion_prom
    FROM sensores
    GROUP BY camion, DATE(fecha_hora)
    ORDER BY camion, dia
""", con, parse_dates=["dia"])
```

`GROUP BY` junta las 24 lecturas de cada día en un solo número por camión.

7. Paso 4: la regla de alerta

Tomamos los primeros 10 días como "normales". Para cada camión calculamos el promedio y el desvío de la vibración diaria, y definimos un umbral: promedio más 3 desvíos. Si un día el promedio supera el umbral, es una alerta.

```
base = diario[diario["dia"] < diario["dia"].min() + pd.Timedelta(days=10)]
stats = base.groupby("camion")["vibracion_prom"].agg(["mean", "std"])
diario = diario.join(stats, on="camion")
diario["umbral"] = diario["mean"] + 3 * diario["std"]
diario["alerta"] = diario["vibracion_prom"] > diario["umbral"]
```

Resultado: solo el camión 2 dispara alertas, desde el 26 de octubre.

8. Paso 5: el gráfico

Una línea por camión y un punto rojo en cada día con alerta:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 5))
for camion, g in diario.groupby("camion"):
    ax.plot(g["dia"], g["vibracion_prom"], label=f"Camión {camion}")
    a = g[g["alerta"]]
    ax.scatter(a["dia"], a["vibracion_prom"], color="red")
ax.set_title("Vibración diaria por camión (datos simulados)")
ax.legend()
fig.savefig("vibracion_camiones.png", dpi=150)
```

9. Cómo correr todo junto

En la carpeta `datos` está el script completo:

```
python demo_mantenimiento_predictivo.py
```

Genera el CSV de datos simulados, la base SQLite y el gráfico en la carpeta `salida`.

10. Qué es y qué no es este ejemplo

- Es una forma simple de ver cómo funciona el mantenimiento predictivo: mirar sensores y avisar antes de la falla.
- No son datos de una mina real, ni de Veladero ni de ninguna otra.
- La regla (promedio más 3 desvíos) es la más básica posible. En la vida real hay más sensores, ruido, falsas alarmas y se validan las alertas con el equipo de mantenimiento.
- El objetivo del mantenimiento predictivo, según describen los medios del sector, es evitar paradas imprevistas y reducir costos de reparación.

11. Cómo seguir

- Cambiá el umbral a 2 o a 4 desvíos y mirá qué pasa con las alertas.
- Usá también la temperatura, no solo la vibración.
- Probá una ventana móvil de 24 horas en lugar del promedio diario.
- Probá un modelo de detección de anomalías de scikit-learn, como `IsolationForest`.
- Reemplazá los datos simulados por datos reales cuando consigas un conjunto público de sensores.

12. Glosario

- **pandas:** librería de Python para trabajar con tablas.
- **SQL:** lenguaje para consultar tablas de datos.
- **SQLite:** base de datos liviana que viene con Python.
- **Desvío estándar:** cuánto se aleja un dato típico del promedio.
- **Anomalía:** un dato que se aleja de lo normal.
- **Mantenimiento predictivo:** usar datos para anticipar una falla y reparar a tiempo.

Fuente de la idea

- Tiempo de San Juan, usos de la IA en minería: https://www.tiempodesanjuan.com/mineria/la-ia-llego-a-las-minas-los-5-usos-clave-que-transforman-la-industria-y-la-clave-academica-en-san-juan_1784564840/amp